

UCS503P – Software Engineering Project

POTLI

On-Demand Luggage Storage Through Trusted Local Businesses

Submitted by:

1024030088 Satyam Tiwari
1024030525 Ishant Mehndiratta
1024030494 Anshaj
1024030498 Aayush Bindal

BE Third Year, Computer Science and Engineering

Submitted to:

Dr. Nisha Thakur

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Abstract

POTLI is a web-based luggage-storage network intended to connect travelers with verified shops, hotels, and other local businesses that have spare storage capacity. The system addresses a practical gap in Indian cities, particularly tier-2 and tier-3 locations, where short-term luggage storage is scarce and difficult to discover. A traveler can locate a nearby partner, reserve capacity, and use a QR code or one-time password (OTP) for an auditable drop-off and pickup. Partners gain an additional income channel, while administrators oversee verification, activity, and service quality.

This mid-semester report documents the problem, requirements, architecture, data flows, UML models, implementation status, and evaluation plan. The current prototype contains a responsive TypeScript/Vite frontend, an Express API with versioned authentication endpoints, Supabase Auth sessions, and a Drizzle-managed Supabase PostgreSQL schema for public user and partner profiles. Search, booking, handoff, payment, rating, and full administrative operations remain designed work and are explicitly distinguished from implemented functionality. The primary proposed evaluation measure is median Booking Fulfillment Time (BFT), targeted at three minutes or less during a pilot.

Contents

Abstract	i
1 Introduction	1
1.1 Background and Context	1
1.2 Problem Statement	1
1.3 Stakeholders	1
1.4 Project Objectives	2
1.5 Scope and Current Status	2
2 Requirements and System Analysis	5
2.1 Functional Requirements	5
2.1.1 Traveler Requirements	5
2.1.2 Partner Requirements	5
2.1.3 Administrator Requirements	5
2.2 Non-Functional Requirements	5
2.3 Use-Case Model	6
2.4 Activity Model	7
3 Methodology and Design	9
3.1 Development Approach	9
3.2 High-Level Architecture	11
3.3 Data Flow Diagrams	11
3.3.1 Level 0: Context	11
3.3.2 Level 1: System Decomposition	12
3.3.3 Level 2: Booking and Handoff	14
3.4 Domain Model	16
3.5 Behavioral Models	17
3.5.1 Sequence Diagram	17
3.5.2 Collaboration Diagram	18
3.5.3 State Diagram	21
4 Implementation Details	23
4.1 Frontend Prototype	23
4.2 Backend Authentication API	23
4.3 Database Schema	24
4.4 Security and Privacy Considerations	24

5	Testing and Evaluation	25
5.1	Testing Methodology	25
5.2	Current Verification Status	25
5.3	Evaluation Metrics	26
5.4	Pilot Plan	26
6	Conclusion	29
6.1	Summary of Work	29
6.2	Key Findings	29
6.3	Future Work and Recommendations	29
6.4	Final Remarks	30
	Bibliography	31

List of Figures

2.1	POTLI use-case diagram	6
2.2	End-to-end POTLI activity diagram	7
3.1	Project development timeline	10
3.2	Level 0 context data flow diagram	12
3.3	Level 1 system data flow diagram	13
3.4	Level 2 booking and luggage-handoff data flow diagram	15
3.5	Planned POTLI domain class diagram	16
3.6	Booking and handoff sequence diagram	18
3.7	Create-booking collaboration diagram	20
3.8	Booking lifecycle state diagram	21

List of Tables

1.1	Implemented and planned project scope	3
5.1	Mid-semester verification status	26
5.2	Proposed evaluation metrics	26

Introduction

1.1 Background and Context

Travelers often face an awkward interval between transport, accommodation, meetings, and sightseeing. During this period, carrying luggage reduces mobility and can make otherwise useful time inconvenient. Formal locker facilities are generally concentrated at major railway stations and airports; they may be full, geographically unsuitable, or difficult to assess in advance. Simultaneously, local businesses may have safe but underused space without a structured mechanism to offer it.

POTLI provides a coordination and trust layer between these groups. Travelers gain discoverable storage, reservation records, and verified handoffs. Storage partners gain tools to publish capacity, working hours, and prices. Platform administrators can review partners, monitor activity, and handle exceptions. The project therefore combines marketplace discovery with a controlled physical handoff workflow rather than acting only as a listing directory.

1.2 Problem Statement

The specific problem is the absence of a dependable, easily discoverable, short-duration luggage-storage service near common urban destinations. Informal arrangements lack consistent identity checks, receipts, handoff evidence, availability information, and dispute records. The engineering challenge is to build a deployable web platform that makes the interaction quick for travelers, simple for small businesses, and auditable for administrators.

The intended deployment environment creates additional constraints: partner devices may be low-to-mid-range smartphones; users may have intermittent connectivity; onboarding must be comprehensible to non-technical business owners; and the software should rely on mature web infrastructure rather than specialized hardware.

1.3 Stakeholders

- **Traveler:** searches, compares locations, books a slot, presents handoff credentials, tracks status, and rates the experience.
- **Storage partner:** registers a business, publishes capacity and hours, confirms drop-off and pickup, and views earnings.

- **Administrator:** verifies partners, audits bookings and fees, reviews reports, and resolves flagged cases.
- **Payment gateway:** processes collections, refunds, payouts, and transaction status in the planned system.

1.4 Project Objectives

1. Deliver a responsive web interface for traveler discovery, account access, partner onboarding, and role-specific operations.
2. Implement secure role-based authentication and a persistent data model for travelers, storage partners, and administrators.
3. Build a booking lifecycle with availability validation, QR/OTP-based drop-off and pickup, timestamps, and photographic handoff evidence.
4. Instrument the workflow so that median BFT and handoff success can be measured objectively.
5. Prepare the system for payment settlement, administrative oversight, repeatable deployment, and later geographic scaling.

1.5 Scope and Current Status

The repository contains both an implemented prototype and design models for later increments. Table 1.1 prevents planned behavior from being mistaken for completed functionality.

Capability	Status	Evidence
Responsive landing and role entry pages	Implemented	TypeScript-rendered landing, traveler/admin login, partner sign-up, and dashboard shells
Versioned authentication API	Implemented	/api/v1/health, /auth/login, and /auth/me
Role-based user persistence	Implemented	Supabase <code>auth.users</code> , <code>public profiles</code> , and <code>partner_profiles</code>
Search and listing availability	Planned	Modeled by DFD process 2.0 and UML interactions
Booking and QR/OTP handoff	Planned	Modeled by DFD processes 3.1–3.5
Payments, payouts, ratings, and disputes	Planned	Included in approved design and later deliverables

Table 1.1: Implemented and planned project scope

Requirements and System Analysis

2.1 Functional Requirements

2.1.1 Traveler Requirements

A traveler should be able to search by city, landmark, or proximity; compare price, opening hours, and availability; reserve a time window; receive a booking identifier and QR/OTP; verify both physical handoffs; review current and prior bookings; and submit a post-pickup rating.

2.1.2 Partner Requirements

A partner should be able to create a business profile, submit details for verification, define capacity and operating hours, receive reservations, validate handoff credentials, attach a drop-off photograph, restore capacity after pickup, and review earnings.

2.1.3 Administrator Requirements

An administrator should be able to approve or reject partner applications, inspect users and listings, monitor booking states, audit payment and platform-fee records, review performance summaries, and investigate disputes or suspicious listings.

2.2 Non-Functional Requirements

- **Usability:** short forms, clear role-specific screens, and a mobile-responsive interface.
- **Security:** salted password hashes, signed expiring tokens, authorization checks, server-side validation, secret management, and protected handoff evidence. Authentication controls should follow established web guidance [OWA25].
- **Reliability:** explicit booking states and idempotent state transitions to avoid duplicate confirmations.
- **Performance:** indexed database queries, pagination, compressed assets, and a median BFT target of at most three minutes.
- **Scalability:** stateless API instances and a database structure that can later support indexed location queries.

- **Maintainability:** versioned endpoints, migration scripts, source-controlled diagram definitions, and clear separation of frontend, API, and data layers.

2.3 Use-Case Model

Figure 2.1 places traveler, partner, and administrator interactions inside one system boundary. It demonstrates that booking depends on discovery and that the trust process continues through both handoffs and administrative oversight.

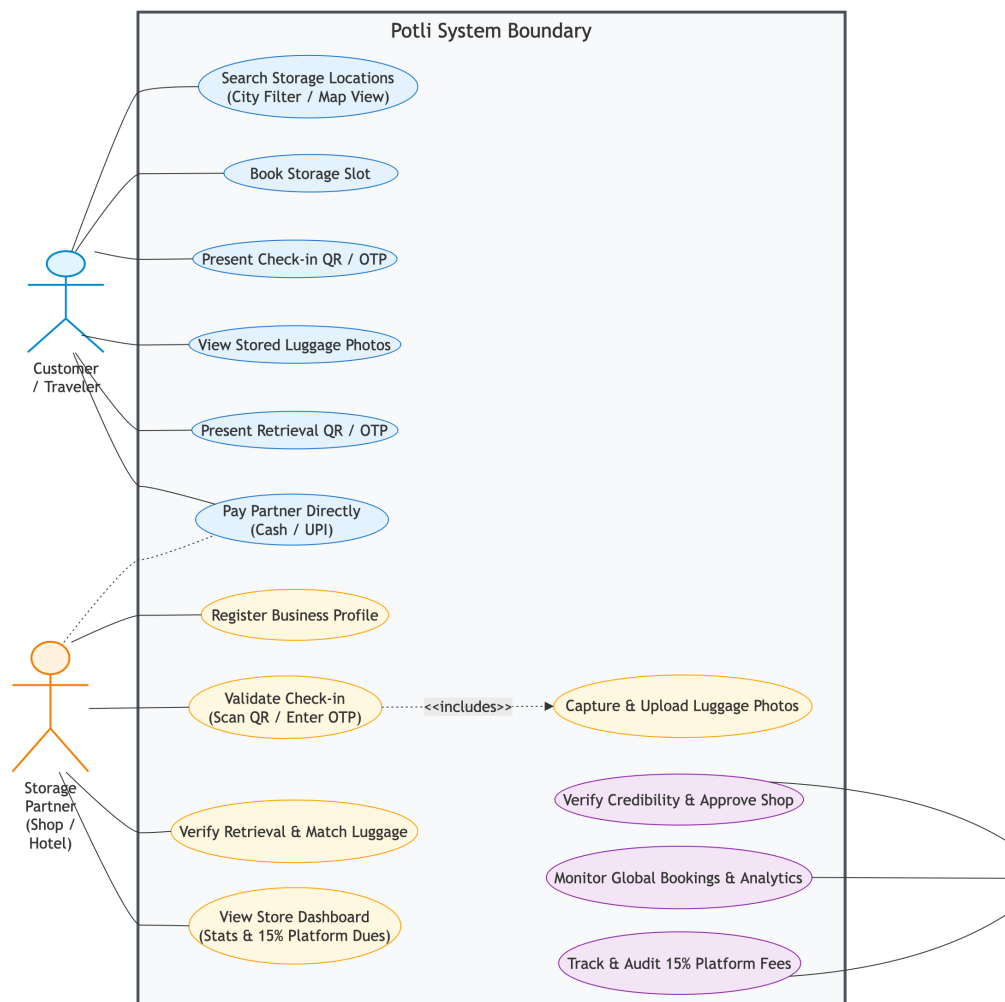


Figure 2.1: POTLI use-case diagram

2.4 Activity Model

The activity diagram in Figure 2.2 follows the complete workflow across traveler, platform, storage partner, and administrator swimlanes. It includes partner approval, discovery, reservation, verification, storage, retrieval, and completion. The model is broader than the current prototype and acts as the implementation roadmap.

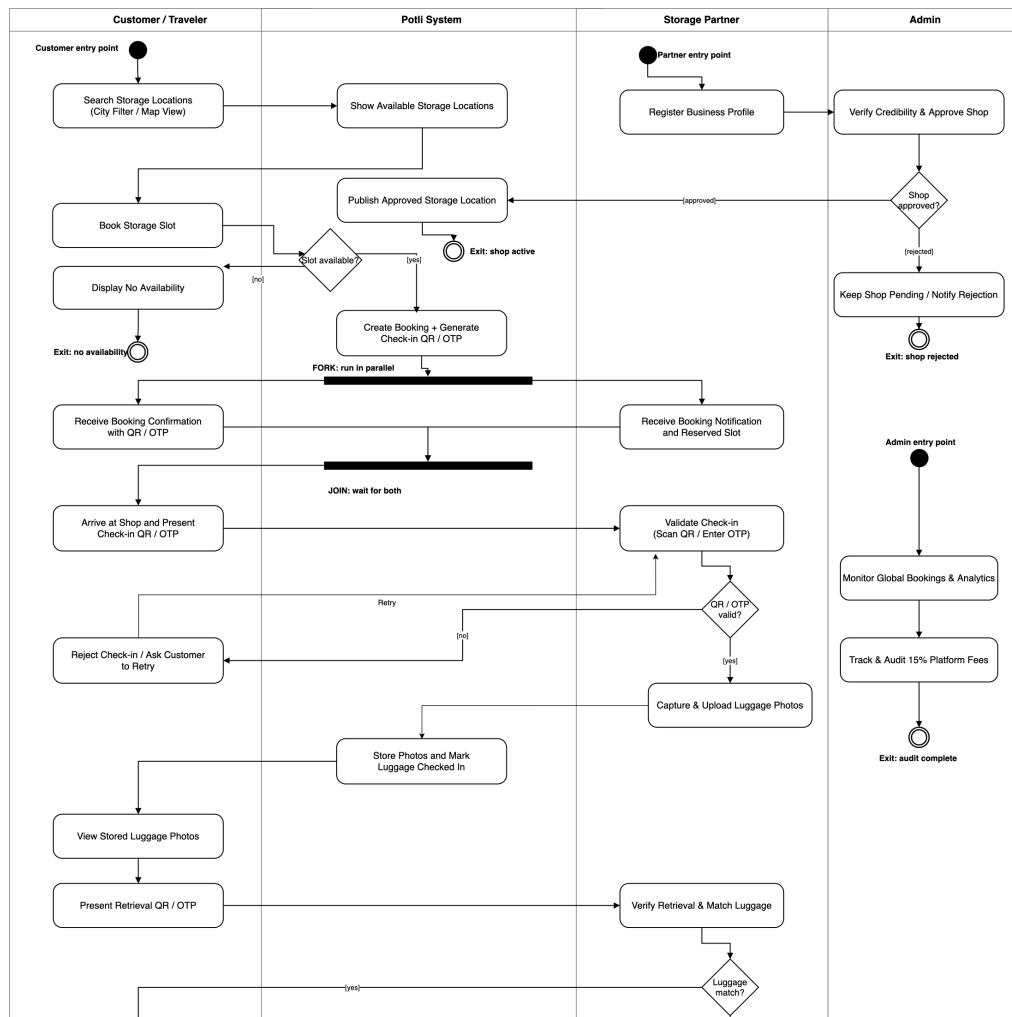


Figure 2.2: End-to-end POTLI activity diagram

Methodology and Design

3.1 Development Approach

The project follows incremental delivery. The foundation increment establishes the interface, account model, authentication API, and database connectivity. Subsequent increments add listings and search, then booking and handoff state transitions, followed by partner/admin operations, payments, testing, and deployment. This sequence provides early demonstrable value while preserving traceability from requirements to implementation.

The Gantt chart in Figure 3.1 organizes the work from requirements through launch demonstration. It should be read as the approved development plan rather than a claim that all activities are complete.

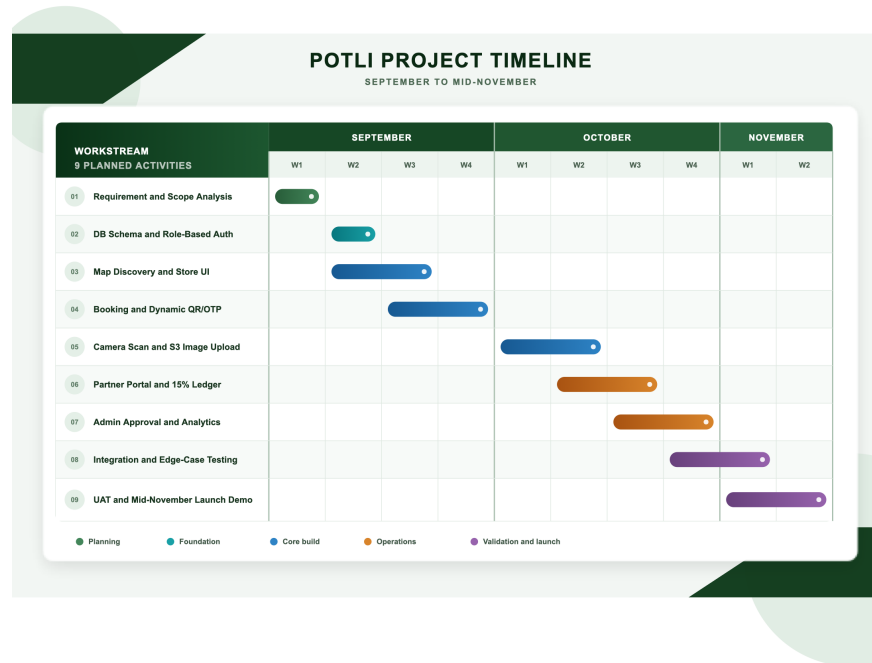


Figure 3.1: Project development timeline

3.2 High-Level Architecture

POTLI uses a three-tier web architecture:

1. **Presentation layer:** a responsive TypeScript application bundled with Vite [Vit25]. It renders route-specific pages and uses the Supabase JavaScript client to persist and refresh browser sessions.
2. **Application layer:** a TypeScript Express [Ope25] API. Implemented endpoints are grouped below `/api/v1`; deprecated unversioned aliases emit deprecation and sunset headers.
3. **Data layer:** Supabase PostgreSQL [Pos25], accessed through a shared Drizzle ORM workspace over the `pg` driver. Supabase Auth owns credentials, while public profile tables hold application roles and partner details.

The browser signs in through Supabase Auth and receives a managed access/refresh-token session. It sends the access token to the API using the Bearer scheme; the API verifies it through Supabase, then loads the server-controlled role from the public profile. The API also exposes server-side login for non-browser clients. Search, booking, media storage, and payment integrations will extend this baseline.

3.3 Data Flow Diagrams

3.3.1 Level 0: Context

The Level 0 DFD (Figure 3.2) treats POTLI as a single process. Registration, search, booking, business updates, administration, and payment data cross the system boundary through four external entities.

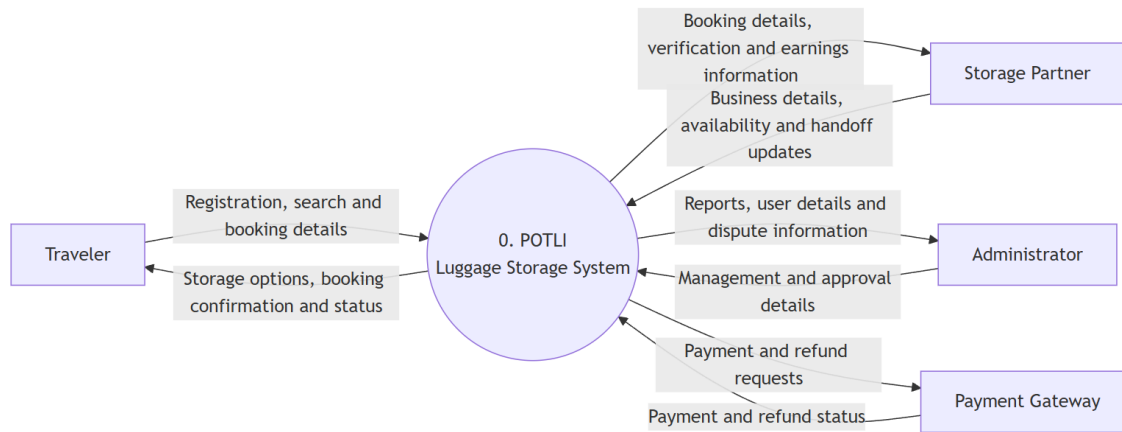


Figure 3.2: Level 0 context data flow diagram

3.3.2 Level 1: System Decomposition

Figure 3.3 decomposes the platform into account management (1.0), storage listing management (2.0), bookings and handoffs (3.0), payment processing (4.0), and platform administration (5.0). Four conceptual stores hold user/partner data, listings, bookings, and payments.

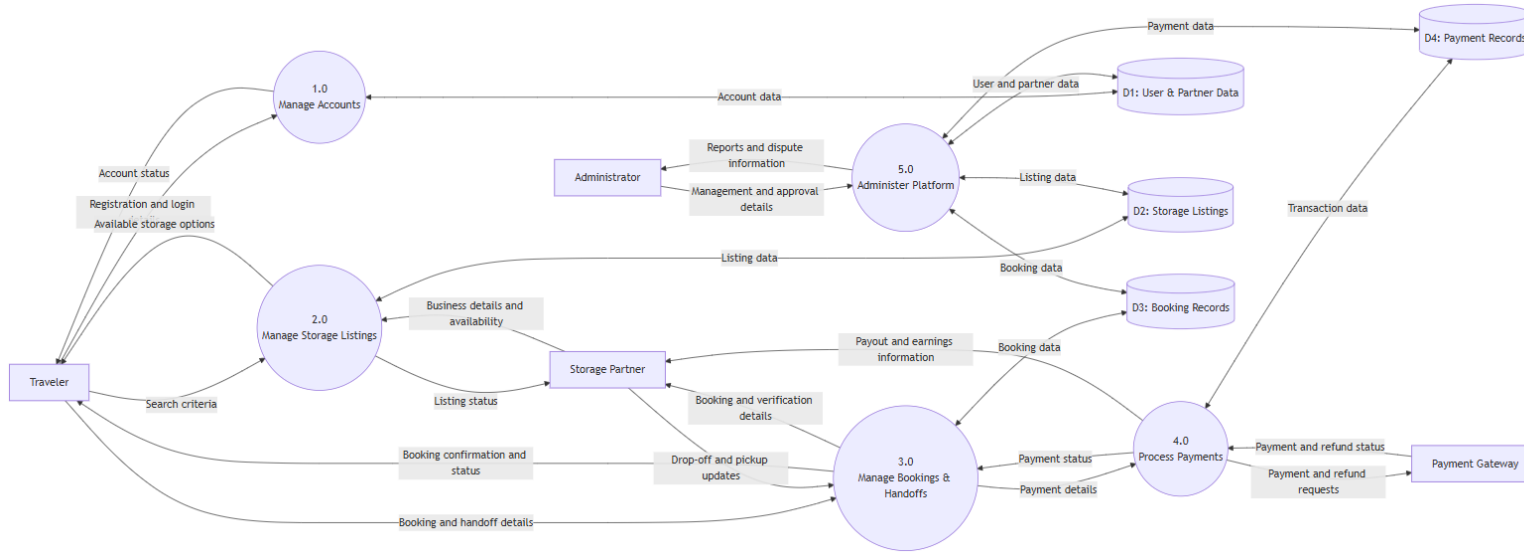


Figure 3.3: Level 1 system data flow diagram

3.3.3 Level 2: Booking and Handoff

The detailed DFD in Figure 3.4 expands process 3.0. A request is validated against listing data, a booking is created after payment status is received, the drop-off is verified and recorded, and pickup completes the booking while restoring capacity.

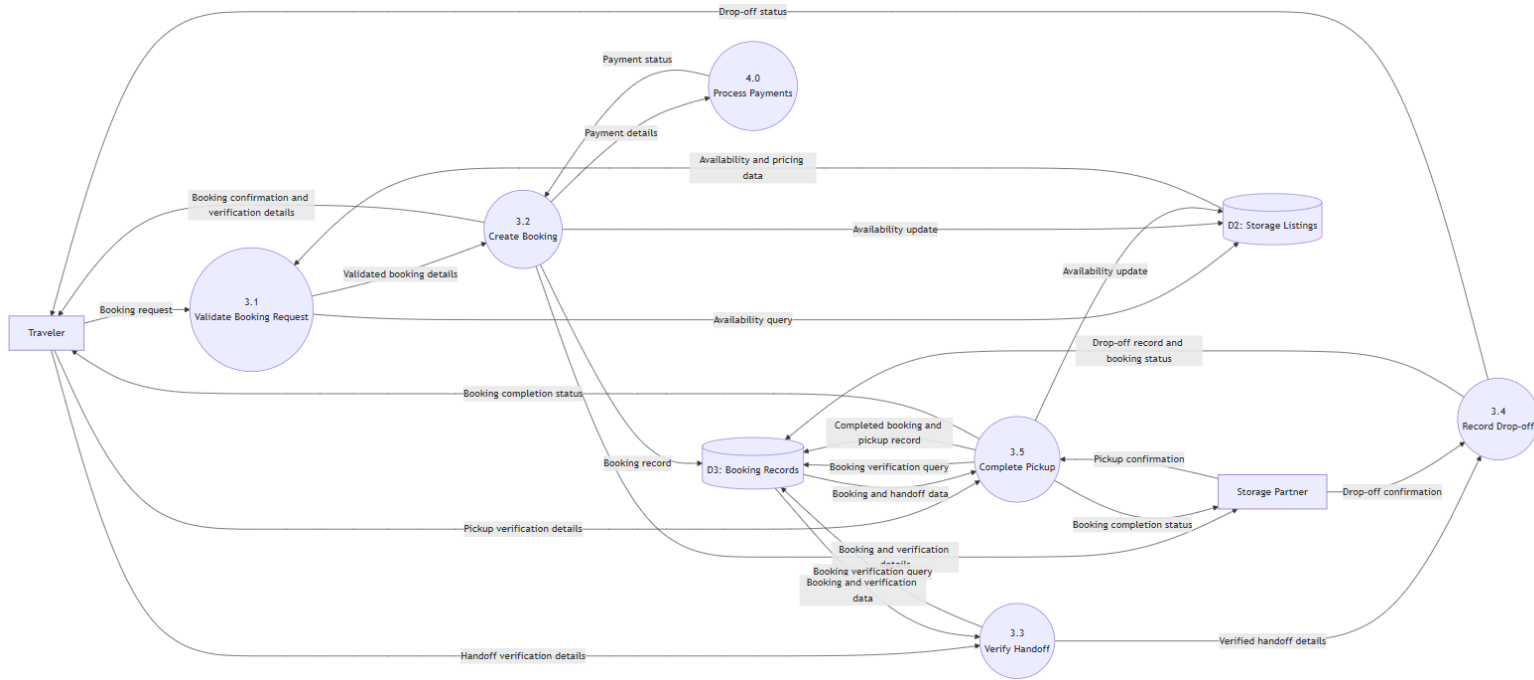


Figure 3.4: Level 2 booking and luggage-handoff data flow diagram

3.4 Domain Model

The class model in Figure 3.5 connects the common **User** identity to traveler, partner, and administrator roles. It separates a partner's business profile from storage listings and composes booking records with handoff and payment information. Ratings remain optional because they occur after successful pickup.



Figure 3.5: Planned POTLI domain class diagram

3.5 Behavioral Models

3.5.1 Sequence Diagram

Figure 3.6 shows the designed end-to-end interaction among traveler, web app, back-end, database, payment gateway, and partner. The implemented login exchange appears within a larger planned sequence for discovery, booking, handoff, settlement, and rating.

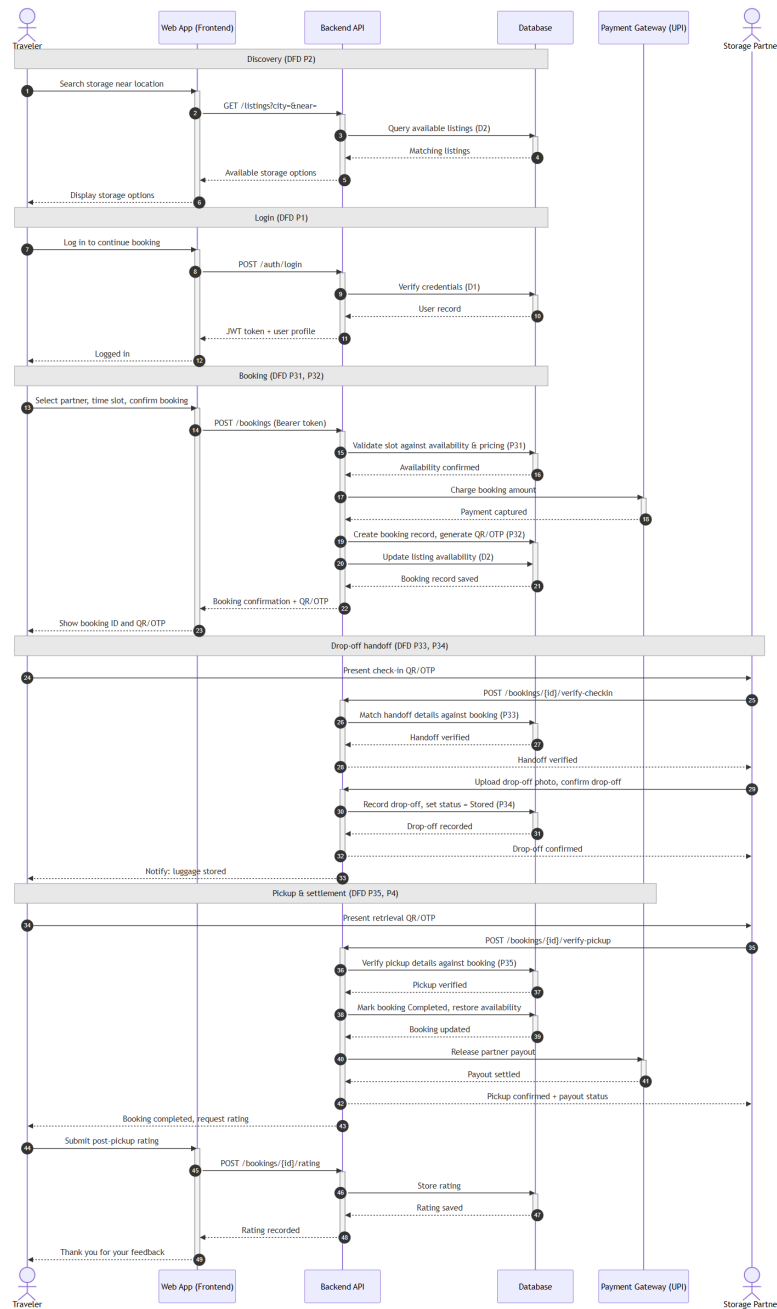


Figure 3.6: Booking and handoff sequence diagram

3.5.2 Collaboration Diagram

The collaboration diagram (Figure 3.7) presents the booking phase as numbered object-to-object messages rather than a time axis. It emphasizes the links among

the web app, booking service, listing store, payment gateway, booking store, and partner.

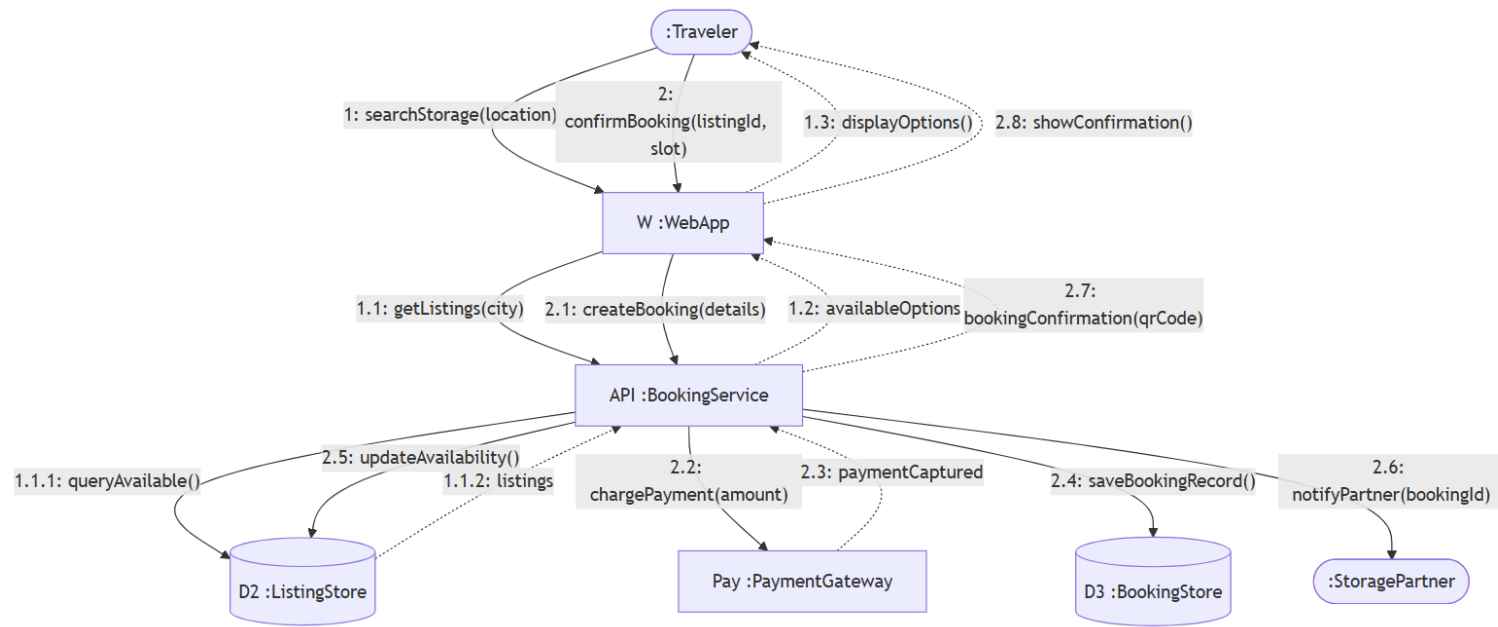


Figure 3.7: Create-booking collaboration diagram

3.5.3 State Diagram

The booking state machine in Figure 3.8 makes valid transitions explicit. A validated request enters **PendingPayment**; successful capture creates **Booked**; verified handoffs lead through **Dropped**, **Stored**, and **PickedUp**; payout settlement produces **Completed**. Failed payment, cancellation, or a missed slot enters a terminal exception state.

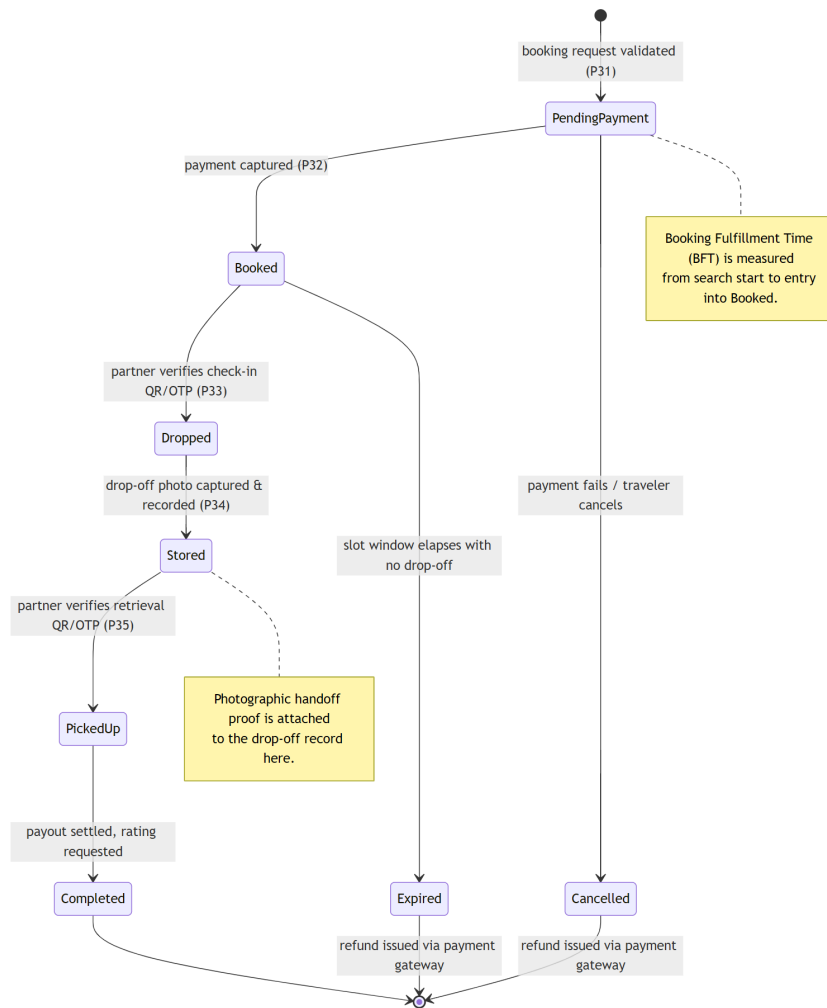


Figure 3.8: Booking lifecycle state diagram

Implementation Details

4.1 Frontend Prototype

The frontend is a Vite project written in TypeScript and styled with a custom responsive stylesheet. A single rendering entry point maps URL hashes to the landing page, traveler login, admin login, partner account page, or role dashboard. Reusable inline SVG icons, semantic forms, labels, status messages, and navigation links reduce external UI dependencies.

The landing experience communicates the service model, trust controls, partner proposition, and project team. Authentication forms use `supabase-js`, which persists and refreshes the session, and then call the versioned backend to load the public role profile. The role dashboard changes its copy for traveler, storage partner, or administrator, and each sign-in route rejects credentials for an incompatible role. Search submission and new partner registration are currently interface demonstrations; they do not yet create persistent domain records.

4.2 Backend Authentication API

The backend uses TypeScript on Node.js and Express with CORS and JSON middleware. Its main implemented responsibilities are:

- checking database availability through `GET /api/v1/health`;
- validating email/password credentials through `POST /api/v1/auth/login`;
- delegating credential checks and token issuance to Supabase Auth;
- verifying a bearer token, loading its public profile, and returning the user through `GET /api/v1/auth/me`; and
- retaining temporary unversioned aliases with explicit deprecation metadata.

Database access is centralized in the shared Drizzle workspace. Credentials and token signing remain owned by Supabase Auth; the backend uses the publishable key to verify tokens and does not require a service-role secret. Asynchronous route failures reach centralized error handling rather than terminating the process.

4.3 Database Schema

Supabase Auth owns `auth.users` and its credential records. A Drizzle-defined public `profiles` table references each auth UUID and stores email, full name, a constrained `traveler/storage_partner/admin` role, and timestamps. The `partner_profiles` table extends partner accounts with business name and phone number using a cascading foreign key. A signup trigger creates traveler profiles, while row-level security allows users to read their own profile and administrators to read all profiles.

Later migrations will introduce the designed D2–D4 stores: listings and capacity, bookings and handoff records, and payment/settlement records. Transactional constraints will be required so simultaneous requests cannot reserve the same final unit of capacity.

4.4 Security and Privacy Considerations

The prototype delegates password hashing and signed sessions to Supabase Auth and protects profile access with row-level security, but a deployable service requires additional controls. These include rate limiting, generic authentication errors, refresh/revocation strategy, secure cookie evaluation in place of unrestricted local storage, authorization middleware for every protected role, strict request schemas, TLS, audit logs, and controlled access to handoff photographs. Personally identifiable information and location history should be minimized and retained only for defined operational periods.

Testing and Evaluation

5.1 Testing Methodology

The planned verification strategy combines:

- **Unit tests** for validation, status transitions, pricing, token handling, and utility functions;
- **API integration tests** against an isolated PostgreSQL database for login, authorization, booking races, handoff codes, and invalid transitions;
- **Frontend tests** for route rendering, form behavior, error handling, responsive layout, and accessibility;
- **End-to-end tests** for traveler booking through partner pickup and administrator review;
- **Performance tests** for nearby search and concurrent booking creation; and
- **Pilot user testing** with representative travelers and local businesses.

5.2 Current Verification Status

At the mid-semester stage, root workspace scripts support full TypeScript checking, a deterministic frontend production build, and an idempotent Supabase migration/seed workflow. The backend does not yet contain an automated test suite, and live API verification depends on a configured PostgreSQL connection. Accordingly, no fabricated accuracy, throughput, uptime, or pilot figures are reported here.

Check	Status	Interpretation
TypeScript compilation and Vite production build	Available	Detects type/build failures and produces deployable static assets
Database migration and seed scripts	Available	Creates development roles when a valid database URL is supplied
Automated backend unit/integration suite	Not yet implemented	Required before release readiness can be claimed
End-to-end booking workflow	Not yet testable	Booking and handoff modules remain planned
Pilot performance measurements	Not yet collected	Must follow implementation and field trial

Table 5.1: Mid-semester verification status

5.3 Evaluation Metrics

Metric	Target	Measurement
Median Booking Fulfillment Time	≤ 3 minutes	Search-start timestamp to confirmed-booking timestamp
Handoff success rate	Establish after pilot	Completed drop-off and pickup without dispute / initiated hand-offs
Partner utilization	Establish baseline	Booked capacity / listed capacity per partner-week
Payout turnaround	Define with gateway	Completion timestamp to credited-payout timestamp
Traveler satisfaction	1–5 scale	Post-pickup rating and short qualitative feedback
Business-hour availability	$\geq 99\%$ pilot goal	Successful health/search probes during defined hours

Table 5.2: Proposed evaluation metrics

5.4 Pilot Plan

A small locality near a railway station or busy market can provide a controlled first deployment. The proposed pilot includes 8–12 partner businesses and 15–20 traveler

participants. Before use, partners receive a brief handoff procedure and the team confirms business details. Application timestamps provide quantitative measures, while structured post-use questions capture convenience, trust, and onboarding difficulty. Any safety incident or disputed handoff takes priority over performance measurement.

Conclusion

6.1 Summary of Work

POTLI defines a practical web platform for connecting travelers who need short-term luggage storage with local businesses that can provide it. The project has progressed from proposal and requirements into a documented architecture, balanced DFDs, UML behavioral and structural models, a responsive frontend prototype, a versioned Express authentication API, and a PostgreSQL role schema.

The current implementation proves the foundation rather than the complete service. Its strongest completed elements are role-aware account access, API/database connectivity, a coherent user experience, and detailed traceable system design. Core transactional capabilities—search-backed listings, reservations, QR/OTP handoffs, payment settlement, ratings, and administrative operations—remain future increments.

6.2 Key Findings

- Trust must be represented as verifiable workflow data: identity review, explicit state transitions, handoff credentials, timestamps, and photographic evidence.
- Availability and booking creation form the central consistency boundary; they must be updated transactionally to prevent overbooking.
- A shared user table with constrained roles simplifies authentication, while separate partner profiles and domain tables preserve extensibility.
- The BFT target is useful only when timestamps and state definitions are implemented before pilot measurement.

6.3 Future Work and Recommendations

1. Implement listing management and indexed location search.
2. Add transactional booking creation and dynamic QR/OTP credentials.
3. Build protected partner handoff screens with secure photo upload.
4. Introduce role-authorization middleware and comprehensive automated tests.

5. Integrate a sandbox payment gateway before enabling real collections or payouts.
6. Build administrator verification, audit, analytics, and dispute tools.
7. Deploy a staging environment, run the proposed pilot, and replace target values with measured results.

6.4 Final Remarks

The project is technically feasible with established web components, but its success depends as much on operational clarity and trust as on interface quality. The next milestone should prioritize one complete, tested path from available listing to verified pickup before broadening features. This vertical slice will make the evaluation metrics meaningful and expose the highest-risk integration points early.

Bibliography

- [Ope25] OpenJS Foundation. *Express: Fast, unopinionated, minimalist web framework for Node.js*. 2025. URL: <https://expressjs.com/> (visited on 10/06/2025) (cit. on p. 11).
- [OWA25] OWASP Foundation. *Authentication Cheat Sheet*. 2025. URL: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (visited on 10/06/2025) (cit. on p. 5).
- [Pos25] PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2025. URL: <https://www.postgresql.org/docs/> (visited on 10/06/2025) (cit. on p. 11).
- [Vit25] Vite Contributors. *Vite: Next Generation Frontend Tooling*. 2025. URL: <https://vite.dev/> (visited on 10/06/2025) (cit. on p. 11).